

Fairly Fair Scheduler Evaluation

Osman Muhina CS 370

1 List of Changes

Below are the changes made to the existing xv6 code base:

- **kernel/proc.h:** Added `rudeness`, `ticksUsed`, and `ticksAllowed` fields to the `struct proc` to support the Fairly Fair scheduling algorithm. Default rudeness is set to 10.
- **kernel/proc.c:**
 - In `allocproc()`, initialized `p->rudeness = 10` for newly allocated processes.
 - Implemented the `rude(int r)` function, which sets the rudeness of the calling process to a passed integer value.
 - Replaced the default Round Robin `scheduler()` loop with the Fairly Fair algorithm: for each scheduling window, the cumulative rudeness R of all active processes is computed, each process is assigned a time quantum $T_i = (r_i \times T)/R$ (with $T = 50$ ticks), and processes are then run in order of `pid` for their assigned quantum. The original Round Robin code was preserved (commented out) for reference and re-testing.
- **kernel/sysproc.c:** Added `sys_rude`, a syscall wrapper that extracts the integer argument via `argint` and calls `rude()`.
- **kernel/syscall.h:** Added a new syscall number, `SYS_rude`.
- **kernel/syscall.c:** Added the `extern` declaration for `sys_rude` and registered it in the syscall dispatch array.
- **kernel/defs.h:** Added function declarations for `rude(int)`.
- **user/user.h:** Added the user-space declaration for `rude(int)` so user programs can call the new syscall.
- **user/usys.pl:** Added an entry to generate the user-space stub for the `rude` syscall.
- **user/evaluate.c:** Implemented the evaluation program. Forks three child processes with rudeness values of 5, 15, and 30, each performing a distinct task on a shared 1000-element array (sum, min-max normalization, and bubble sort respectively). Each child records its own turnaround time using `uptime()` before and after its task, and the parent records total elapsed time across all three children.
- **Makefile:** Added `$U/_evaluate` to `UPROGS` so the evaluation program is built and available in the xv6 shell.

2 Experiment

I believe that turnaround time will decrease under the Fairly Fair scheduler compared to Round Robin, particularly for processes with higher rudeness values. I believe this because the Fairly Fair algorithm allocates CPU time proportionally to a process's rudeness value, meaning higher-rudeness processes (such as

the bubble sort task with rudeness 30) receive larger time quanta per scheduling window and should therefore complete their work in less total wall-clock time than under Round Robin’s equal time-slicing approach.

Average turnaround time (across all three child processes) was measured across 3 runs of both the Round Robin and Fairly Fair approaches. The measurements and averages are reported in the table below. Turnaround time is measured in ticks.

Round Robin	Fairly Fair
647.0	465.7
443.7	676.7
452.7	365.7
Avg: 514.5	Avg: 502.7

Table 1: Average turnaround time (ticks) across 3 children, per run

2.1 Results

From the results, I tentatively accept my hypothesis, though the improvement was modest. The average turnaround time decreased slightly under the Fairly Fair scheduler (502.7 ticks) compared to the baseline Round Robin approach (514.5 ticks). However, the difference is small relative to the run-to-run variance observed within each scheduler (e.g., Fairly Fair ranged from 365.7 to 676.7 across runs), suggesting that with only three trials per scheduler, this result should be interpreted cautiously rather than as strong confirmation. Notably, Child 0 (rudeness 5, the sum task) consistently finished in near 0 ticks across nearly all runs regardless of scheduler, since its workload was too small to meaningfully benefit from either approach’s CPU allocation strategy. The more interesting comparisons are between Child 1 and Child 2, whose relative finishing order varied between runs, indicating that with only three discrete processes competing for time slices, the proportional allocation of the Fairly Fair scheduler did not consistently or dramatically outperform Round Robin at this scale.